Name of the Student	**AKHILA POCHIMIREDDY**
Reg. No	**192425225**
GitHub Link	**https://github.com/120608akhila/MLA0405-DEEP-LEARNING/tree/main/Assessment/CO3**

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Game Based Learning

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0405**

SLOT :D

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25% **Duration: 1 Hour**

Assessment Tool Weightage: 35% **Marks: 50**

Code of Conduct:

I **AKHILA POCHIMIREDDY(192425225)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Akhila.P **Name of the student:** P.Akhila

---

Criteria	Understanding of Concept **(2.5)**	Conceptual Clarity **(2.5)**	Problem-Solving Ability **(2)**	Solution Design & Application **(2)**	Teamwork & Game Participation ***(1)***	**Total (10)**
Weightage	***25 %***	***25%***	***20%***	***20 %***	***10 %***	**100%**
Round 1						
Round 2						
Total						

Signature of the Faculty: ***Total Marks :***

QUESTIONS:01 **Total Marks: 50**

Question:

Design and participate in a two-round game-based learning activity to explore the fundamental concepts of Convolutional Neural Networks (CNNs), including architecture, layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world applications. Progress through five levels in each round, applying your knowledge to solve challenges involving CNN design, feature extraction, architecture comparison, and practical applications.

The Game Progression
Round 1:
Basic CNN Concepts → Filters → Parameter Sharing → Architecture → AlexNet vs ResNet
Round 2:
Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

ROUND 1: CNN Architecture Builder (Learn by constructing and understanding CNN architectures layer by layer)

Level	Challenge	Description
Level 1	**CNN Layer Matcher**	Match CNN layers such as Convolution, Pooling, Flatten, and Fully Connected layers with their respective functions and roles in the network.
Level 2	**Filter Finder**	Identify suitable filters and kernels for feature extraction and understand how filter size, stride, and padding affect the output feature map.
Level 3	**Parameter Sharing Puzzle**	Determine how the same filter weights are reused across different image locations and calculate the number of parameters in a convolutional layer.
Level 4	**CNN Architecture Debugger**	Identify and correct problems in a CNN architecture, such as incorrect layer order, incompatible dimensions, and inappropriate layer selection.
Level 5	**AlexNet vs ResNet Challenge**	Compare AlexNet and ResNet architectures and identify how their architectural approaches differ, particularly the use of residual connections in ResNet.

Level 1 – CNN Layer Matcher

Challenge: Match CNN layers with their functions.

Layer	Function
Convolution	Extracts features such as edges, textures and patterns
Pooling	Reduces feature-map size and computation
Flatten	Converts feature maps into a 1D vector
Fully Connected	Performs final classification

Answer:
The correct sequence is generally:

Input Image → Convolution → Pooling → Flatten → Fully Connected → Output

Explanation: Convolution extracts useful features, pooling reduces dimensions, flatten converts the features into a vector, and the fully connected layer uses these features for classification.

Insight: Each layer has a specific role, and using them in the correct order allows the CNN to progressively learn useful image representations.

Level 2 – Filter Finder

Challenge: Identify suitable filters and understand the effect of filter size, stride and padding.

A filter/kernel is a small matrix that moves across an image and performs convolution.

For example, a 3 × 3 filter can be used for detecting edges.

The output size is calculated as:

$$\text{Output} = \frac{\text{N} - \text{F} + 2\text{P}}{S} + 1$$

where:

- N = input size
- F = filter size
- P = padding
- S = stride

Example:
For a 28 × 28 image with a 3 × 3 filter, stride 1 and no padding:

$$\frac{28 - 3}{1} + 1 = 26$$

So, the output feature map is **26 × 26.**

Insight: Smaller filters such as 3 × 3 can detect local features efficiently, while stride and padding control the size of the resulting feature map.

Level 3 – Parameter Sharing Puzzle

Challenge: Understand how the same filter weights are reused and calculate convolution parameters.

In CNNs, the same filter weights are reused at different locations of an image. This is called parameter sharing.

For a convolution layer:

$$\text{Parameters} = (F_{h^*} F_{w^*} C_{in} + 1) * C_0 t$$

Example:
If a layer has:

- Filter = 3 × 3
- Input channels = 3
- Number of filters = 16

Then:

(3×3×3+1)×16 =(27+1)×16=448

So, the layer has **448 trainable parameters**.

Insight: Parameter sharing greatly reduces the number of weights compared with a fully connected network and makes CNNs efficient for image processing.

Level 4 – CNN Architecture Debugger

Challenge: Identify incorrect layer order, incompatible dimensions and inappropriate layer selection.

A valid CNN architecture can be:

Input → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected → Output

Possible errors include:

- Placing Flatten before convolution/pooling.
- Giving an incorrect input dimension.
- Connecting incompatible feature-map dimensions.
- Using an inappropriate layer for the task.
- Forgetting the final classification layer.

Correction: Convolution and pooling should normally be used for feature extraction before flattening and classification.

Insight: Correct layer ordering is important because each layer expects a particular type and dimension of input.

Level 5 – AlexNet vs ResNet Challenge

Challenge: Compare AlexNet and ResNet, especially residual connections.

Feature	AlexNet	ResNet
Depth	Relatively shallow	Much deeper
Structure	Sequential CNN	Residual blocks
Skip connection	No	Yes
Feature learning	Good	More powerful
Deep training	More difficult	Easier

AlexNet uses convolution, ReLU, pooling and fully connected layers.

ResNet introduces **residual/skip connections,** allowing information and gradients to pass more easily through deep networks.

A residual block can be represented as:

$$Output = F(x) + x$$

Conclusion: ResNet is more suitable for complex image-recognition tasks because it can effectively train much deeper networks.

ROUND 2: CNN Feature Detective (Learn by exploring how CNNs extract features, avoid overfitting, and solve real-world problems)

Level	Challenge	Description
Level 1	**Edge Detection Explorer**	Apply simple (3\times3) filters to an image and determine how convolution operations can detect edges and other basic features.

Level	Challenge	Description
Level 2	**Feature Map Detective**	Analyze feature maps produced by different CNN layers and identify the progression from simple features such as edges to complex patterns and objects.
Level 3	**Regularization Rescuer**	Identify overfitting in a CNN model and select suitable regularization techniques to improve generalization and model performance.
Level 4	**CNN Architecture Race**	Compare ResNet and AlexNet for different image-classification applications and determine which architecture is more suitable for a given problem.
Level 5	**CNN Application Master**	Design a CNN-based solution for a real-world application such as medical image analysis, face recognition, object detection, or autonomous driving, and justify the selected architecture and layers.

Level 1 – Edge Detection Explorer

Challenge: Apply a 3 × 3 filter to detect edges.

A convolution filter can detect edges by emphasizing differences between neighboring pixels.

For example, a vertical edge filter can be represented as:

$$\begin{matrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{matrix}$$

When this filter moves across an image, it produces strong values where there is a vertical intensity change.

Explanation:
If neighboring pixels have similar values, the output is small. If there is a strong change in intensity, the output becomes large, indicating an edge.

Insight: CNNs can automatically learn useful edge and pattern detectors through convolution filters.

Level 2 – Feature Map Detective

Challenge: Analyze feature maps from different CNN layers and identify the progression from simple to complex features.

CNNs learn features hierarchically.

CNN Layer	Features Learned
Early layers	Edges and lines
Middle layers	Textures and shapes
Deep layers	Complex patterns and object parts

For example, in a face-recognition system:

Early layer → edges → eyes/nose shapes → facial structures → complete face features

Critical Insight: As depth increases, the features become more abstract and task-specific. This hierarchical feature extraction is one of the main strengths of CNNs.

Level 3 – Regularization Rescuer

Challenge: Identify overfitting and select suitable regularization methods.

If a CNN has **very high training accuracy but low validation/test accuracy**, it is likely suffering from **overfitting**.

Suitable techniques include:

1. Dropout:
Randomly disables some neurons during training, reducing dependency on particular neurons.

2. Data Augmentation:
Creates variations of training images using rotation, flipping, cropping and scaling.

3. Batch Normalization:
Normalizes activations and can improve training stability.

Best approach:
Using **data augmentation + dropout + batch normalization** can improve generalization.

Insight: The goal is not only high training accuracy but also good performance on unseen data.

Level 4 – CNN Architecture Race

Challenge: Select between ResNet and AlexNet for different image-classification applications.

Application	Suitable Architecture	Reason
Simple image classification	AlexNet	Simpler architecture
Complex image recognition	ResNet	Deeper feature learning
Large-scale classification	ResNet	Stronger deep representations
Learning basic CNN concepts	AlexNet	Easier to understand

Decision:
For a complex image-classification problem, **ResNet** is generally the better choice because residual connections allow deeper networks to be trained effectively.

Critical Insight: Architecture selection should depend on the complexity of the problem, computational resources and required performance.

Level 5 – CNN Application Master

Challenge: Design a CNN solution for a real-world application and justify the architecture and layers.

Example: Medical X-Ray Classification

Problem: Classify X-ray images as normal or abnormal.

Proposed CNN

X-ray Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected → Output

Working

1. **Input:** X-ray image is given to the CNN.
2. **Convolution:** Extracts edges, textures and medical patterns.
3. **ReLU:** Adds non-linearity.
4. **Pooling:** Reduces spatial dimensions.
5. **Deeper convolution:** Learns complex abnormal structures.
6. **Flatten:** Converts feature maps into a vector.
7. **Fully Connected:** Performs classification.
8. **Output:** Predicts normal or abnormal.

Justification

CNN is suitable because medical images contain important **spatial patterns**. Convolution layers can automatically learn these patterns without manually designing features.

Conclusion: A CNN provides an effective solution for automated medical image classification and can assist medical professionals in image-based analysis.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand